

- **1. O(N²) Memory Overhead:** Using `+=` in `ConcatString` creates excessive allocations; strings are immutable in Go.
- **2. Missing Pre-allocation:** `BuilderString` should call `builder.Grow(n)` before the loop to prevent internal buffer reallocations.
- **3. Dead Code Risk:** The compiler might skip the benchmark loops entirely because the function returns are unused. Assign results to a global variable.
- **4. Reinventing the Wheel:** Production code should just use `strings.Repeat("a", n)`, which is highly optimized.
- **5. No Correctness Tests:** There are no `Test...` functions to verify the functions actually return the right string.
- **6. Static Benchmark Scope:** Hardcoding `1000` hides performance scaling; benchmarks should test multiple sizes (N=10, 1000, 100000).
- **7. Hardcoded Magic Strings:** `"a"` is hardcoded; the functions should accept the target string as an argument.
- **8. Unhandled Returns:** `builder.WriteString` returns `(int, error)` which should be explicitly ignored (e.g., `_, _ = builder.WriteString(...)`) to satisfy strict linters.
- **9. Missing Package Docs:** The file lacks a standard package-level documentation comment.
- **10. Undocumented Edge Cases:** Behavior for negative `n` inputs (silently returning an empty string) is neither checked nor documented.